

Hilfsmittel zum Modul Programmierung 2

Befehl / Ausdruck	Erläuterung
<code>type(<variable>)</code>	Gibt den Datentyp einer Variablen oder eines Werts zurück
<code>int(47.5) / float(3) / str(123)</code>	Typumwandlung (Casting) auf den gewünschten Datentyp
<code>print("text")</code>	Gibt den Text auf der Konsole aus
<code>i // j</code>	Ganzzahldivision (ohne Rest)
<code>i % j</code>	Modulo-Operator: Gibt den Divisionsrest zurück
<code>i ** j</code>	Potenzierung: i hoch j
<code>"HST"[1] → "S"</code>	Gibt das Zeichen an Index 1 zurück
<code>"HST"[-1] → "T"</code>	Gibt das letzte Zeichen zurück
<code>"HST"[1:2] → "S"</code>	Substring vom Index 1 bis 2 (exkl.)
<code>"HST"[:2] → "HS"</code>	Substring vom Start bis Index 2 (exkl.)
<code>"HST"[1:] → "ST"</code>	Substring ab Index 1 bis zum Ende
<code>"HST"[:] → "HST"</code>	Kopie des gesamten Strings
<code>"TUAS"[1:4:2] → "US"</code>	Substring mit Start:Stop:Step
<code>len("HST") → 3</code>	Gibt die Länge des Strings zurück
<code>3 * "hallo" → "hallohallohallo"</code>	Wiederholt den String 3-mal
<code>"schule" in "Hochschule" → True</code>	Prüft, ob Teilstring enthalten ist
<code>"Apfel,Banane,Kirsche".split(",")</code> <code>" Hallo Welt! \n".strip()</code>	<code>split</code> ergibt: ['Apfel', 'Banane', 'Kirsche'] <code>strip</code> ergebnis: "Hallo Welt!"
<code>hst.count('hst')</code>	Zählt, wie oft 'hst' in hst vorkommt
<code>while <Bedingung>:... break / continue</code>	Schleife mit Abbruch oder Überspringen durch <code>break / continue</code>
<code>for <var> in range(...):... break / continue</code>	For-Schleife mit <code>range()</code> , mit <code>break / continue</code>
<code>range(start, stop, step)</code>	Erzeugt Zahlenfolge von start bis stop-1 in Schritten von step
<code>if char.isnumeric():</code>	Prüft, ob der String nur Ziffern enthält
<code>elif char.isalpha():</code>	Prüft, ob der String nur Buchstaben enthält
<code>bearbeiteter_text[::-1]</code>	Gibt den String rückwärts aus (Slicing)
<code>def beliebig(*args):</code>	Funktion mit beliebig vielen Argumenten
<code>range(1, breite + 1, 2)</code>	Start bei 1, bis breite (inklusive), Schritte +2
<code>for i in reversed(wort)</code>	Gibt die Zeichen von wort rückwärts aus
<code>'texttext'.count('probe')</code>	Zählt, wie oft der Teilstring vorkommt
<code>import random</code> <code>random.randint(1,10)</code>	Zufallszahl zwischen 1 und 10
<code>random.choice(<Liste>)</code>	Wählt zufälliges Element aus einer Liste
<code>import stringabc = string.ascii_letters</code>	Importiert das Alphabet
<code>abc.index(buchstabe)</code>	Gibt den Index des Buchstabens in abc zurück
<code>"Hochschule Trier".split(" ")</code>	Teilt den String in Wörter (Liste)
<code>liste.append("...")</code>	Fügt Element ans Listenende
<code>setname.add("...")</code>	Fügt Element zum Set hinzu
<code>liste.extend([...])</code>	Hängt mehrere Elemente an eine Liste an
<code>setname.update(anderer_set)</code>	Fügt alle Elemente eines Sets hinzu
<code>liste.pop(index)del liste[index]</code>	Entfernt Element an Index
<code>setname.discard("...")</code>	Entfernt Element aus Set (kein Fehler bei Nichtvorhandensein)
<code>liste.index(element)</code>	Gibt den Index eines Listenelements zurück
<code>liste2.extend(liste1)</code>	Hängt liste1 an liste2 an
<code>SET = {...}SET.update()SET.remove()</code>	Set-Definition, Hinzufügen, Entfernen
<code>dictionary = {'...':'...'} dictionary['Color'] = 'green'del dictionary['Color']</code>	Dictionary-Definition, Hinzufügen, Löschen
<code>tuple = ('...', '...')</code>	Tupel-Definition
<code>s.replace('h', 'l')</code>	Ersetzt Zeichen im String
<code>dictionary[i].get('Value')</code>	Gibt Wert zu Schlüssel zurück (oder None)
Testen und Debugging	
<code>pytest aufgabe01.py</code>	Test mit pytest ausführen
<code>assert Datei.Funktion(...) == ...</code>	Überprüft das erwartete Ergebnis
Argumente Kommandozeilenbefehle	
<code>python file.py -a -b ...</code>	Skript mit Kommandozeilenargumenten starten
<code>getopt.getopt(args, shortopts, longopts)</code>	Argument-Parser für Kommandozeile
Datei Handling	
<code>open('file.txt', 'r')</code> bzw. <code>open(infile, 'r', encoding='utf-8') as file</code>	Datei öffnen (z.B. zum Lesen)
<code>f.read() / f.read(x)</code>	Liest gesamte Datei / x Zeichen
<code>f.readline() / f.readlines()</code>	Liest eine / alle Zeilen
<code>f.write() / f.writelines()</code>	Schreiben in Datei
<code>"Zeile1.\nZeile2."</code>	Zeilensprung
<code>f.close()</code>	Datei schließen
<code>ls -la</code>	Zeigt Dateirechte (Linux)
<code>textfile.read().replace(...).split()</code>	Entfernen von Satzzeichen & Aufteilen in Liste
Datenstrukturen / Sortiervverfahren	
<code>numbers[j], numbers[j+1] = numbers[j+1], numbers[j]</code>	Vertauschen von Listenelementen